

Experiment 5

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	Semester	III
Course	Skill Lab: Python Programming	Course Code	VDCOR2VS201
Class	SY VLSI	Course In-Charge	Mr. Nirmol Munvar
Exp Name	List and Dictionary Comprehensions		
CO	Analyze and implement control structures and user-defined functions to solve logical and numerical problems using Python.		

List Comprehension

List comprehension is a concise way of creating a new list from an existing iterable such as a list, tuple, string, or range().

It combines a for loop and optionally a condition into a single expression.

Basic syntax:

```
new_list = [expression for item in iterable]
```

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = [x * x for x in numbers]
```

```
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

The equivalent code using a normal for loop is:

```
squares = []
```

```
for x in numbers:
```

```
    squares.append(x * x)
```

List comprehension can also contain a condition.

Syntax:

```
new_list = [expression for item in iterable if condition]
```

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = [x for x in numbers if x % 2 == 0]
```

```
print(even_numbers)
```

Output:

```
[2, 4, 6]
```

It can also use if-else to determine the value that should be placed in the new list.

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
result = ["Even" if x % 2 == 0 else "Odd" for x in numbers]
```

```
print(result)
```

Output:

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

List comprehensions are particularly useful when the operation is simple and can be expressed clearly in one line.

Dictionary Comprehension

Dictionary comprehension is a concise way of creating a new dictionary from an iterable.

Basic syntax:

```
new_dict = {key: value for item in iterable}
```

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = {x: x * x for x in numbers}
```

```
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Dictionary comprehension can also include a condition.

Syntax:

```
new_dict = {key: value for item in iterable if condition}
```

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_squares = {x: x * x for x in numbers if x % 2 == 0}
```

```
print(even_squares)
```

Output:

```
{2: 4, 4: 16, 6: 36}
```

Dictionary comprehension can also be used to transform an existing dictionary.

```
prices = {"Pen": 10, "Book": 50, "Bag": 500}
```

```
discounted = {item: price * 0.9 for item, price in prices.items()}
```

```
print(discounted)
```

Here, `.items()` provides both the key and value during each iteration.

List comprehension produces a **list**, whereas dictionary comprehension produces a **dictionary containing key-value pairs**.

Basic Exercises

Exercise 1: Number List Processing

Given a list of integers, use **list comprehension** to create:

- A list containing the squares of all numbers.
- A list containing only even numbers.
- A list containing "Positive" or "Negative" for each number.

Exercise 2: Student Marks

Given a list containing the marks of 10 students, use **list comprehension** to create:

- A list containing only marks greater than or equal to 50.
- A list containing the marks after adding 5 grace marks.

- A list containing "Pass" or "Fail" for each student.

Exercise 3: Dictionary of Squares

Given a list of numbers, use **dictionary comprehension** to create a dictionary where each number is the key and its square is the value.

Then create another dictionary containing only the numbers whose squares are even.

Exercise 4: Product Prices

Given a dictionary containing product names and their prices, use **dictionary comprehension** to:

- Create a dictionary containing products costing more than ₹500.
- Create a dictionary containing the same products with a 10% discount.
- Create a list using **list comprehension** containing the names of products costing ₹500 or less.

```
Day_3 > exp_5.py > ...
1 print("\n" * 50)
2 print("EXERCISE 1: Number List Processing")
3 print("\n" * 50)
4
5 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
6 print(f"Original list: {numbers}")
7 print(f"Squares: {[num**2 for num in numbers]}")
8 print(f"Even numbers: {[num for num in numbers if num % 2 == 0]}")
9 print(f"Signs: [{'Positive' if num >= 0 else 'Negative' for num in numbers}]")
10
11 print("\n" * 50)
12 print("EXERCISE 2: Student Marks")
13 print("\n" * 50)
14
15 marks = [45, 67, 89, 34, 78, 56, 91, 48, 62, 73]
16 print(f"Original marks: {marks}")
17 print(f"Marks >= 50: {[mark for mark in marks if mark >= 50]}")
18 print(f"Marks with grace (+5): {[mark + 5 for mark in marks]}")
19 print(f"Results: [{'Pass' if mark >= 50 else 'Fail' for mark in marks}]")
20
21 print("\n" * 50)
22 print("EXERCISE 3: Dictionary of Squares")
23 print("\n" * 50)
24
25 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
26 square_dict = {num: num**2 for num in numbers}
27 print(f"Square dictionary: {square_dict}")
28 even_squares = {num: num**2 for num in numbers if (num**2) % 2 == 0}
29 print(f"Even squares dictionary: {even_squares}")
30
31 print("\n" * 50)
32 print("EXERCISE 4: Product Prices")
33 print("\n" * 50)
34
35 products = {
36     "Notebook": 45, "Pen": 15, "Laptop": 45000,
37     "Mouse": 350, "Keyboard": 1200, "Headphones": 800,
38 }
```

```
31 print("\n" * 50)
32 print("EXERCISE 4: Product Prices")
33 print("\n" * 50)
34
35 products = {
36     "Notebook": 45, "Pen": 15, "Laptop": 45000,
37     "Mouse": 350, "Keyboard": 1200, "Headphones": 800,
38     "USB Drive": 500, "Monitor": 15000, "Desk Lamp": 400,
39     "Charger": 250
40 }
41 print(f"Original products: {products}")
42 print(f"Products > ₹500: {[k: v for k, v in products.items() if v > 500]}")
43 print(f"With 10% discount: {[k: v*0.9 for k, v in products.items()]}")
44 print(f"Products ≤ ₹500: {[k for k, v in products.items() if v <= 500]}")
```

```
PS G:\My Drive\Skill Lab Python Programming> python -u "g:\My Drive\Skill Lab Python Programming\Day_3\exp_5.py"
=====
EXERCISE 1: Number List Processing
=====
Original list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even numbers: [2, 4, 6, 8, 10]
Signs: ['Positive', 'Positive', 'Positive', 'Positive', 'Positive', 'Positive', 'Positive', 'Positive', 'Positive', 'Positive']

=====
EXERCISE 2: Student Marks
=====
Original marks: [45, 67, 89, 34, 78, 56, 91, 48, 62, 73]
Marks >= 50: [67, 89, 78, 56, 91, 62, 73]
Marks with grace (+5): [50, 72, 94, 39, 83, 61, 96, 53, 67, 78]
Results: ['Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Pass', 'Pass', 'Fail', 'Pass', 'Pass']

=====
EXERCISE 3: Dictionary of Squares
=====
Square dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
Even squares dictionary: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

=====
EXERCISE 4: Product Prices
=====
Original products: {'Notebook': 45, 'Pen': 15, 'Laptop': 45000, 'Mouse': 350, 'Keyboard': 1200, 'Headphones': 800, 'USB Drive': 500, 'Monitor': 15000, 'Desk Lamp': 400, 'Charger': 250}
Products > ₹500: {'Laptop': 45000, 'Keyboard': 1200, 'Headphones': 800, 'Monitor': 15000}
With 10% discount: {'Notebook': 40.5, 'Pen': 13.5, 'Laptop': 40500.0, 'Mouse': 315.0, 'Keyboard': 1080.0, 'Headphones': 720.0, 'USB Drive': 450.0, 'Monitor': 13500.0, 'Desk Lamp': 360.0, 'Charger': 225.0}
Products ≤ ₹500: ['Notebook', 'Pen', 'Mouse', 'USB Drive', 'Desk Lamp', 'Charger']
PS G:\My Drive\Skill Lab Python Programming>
```